



Universidad
Técnica de
Manabí
desde 1952

Unidad 1

NOMBRES DEL ESTUDIANTES:

Beltran Anchundia Carlos Adrian

PARALELO:

"A"

ASIGNATURA:

Desarrollo de sistemas informáticos

DOCENTE:

Ing. Jorge Arun Zambrano Cedeño

TEMA:

Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

CÉDULA:

092971672-8

CORREO ELECTRONICO:

Cbeltra6728@utm.edu.ec





En esta actividad usted deberá **diseñar la arquitectura lógica de un sistema de gestión técnica profesional, integrando estándares de modelado UML, aplicación de patrones de diseño y el uso de herramientas de control de versiones (GitHub/Gitflow).**

Objetivo

Diseñar la arquitectura de un sistema de software aplicando patrones de diseño y flujos de trabajo profesionales, fortaleciendo la capacidad de análisis y las buenas prácticas de ingeniería de software para soluciones escalables y mantenibles.

Instrucciones

- Realice el análisis del escenario técnico (identificación de actores y requerimientos).
- Modele el sistema mediante diagramas de casos de uso y clases bajo estándares UML.
- Implemente al menos tres patrones de diseño (Creacionales, Estructurales o de Comportamiento).
- Utilice el flujo de trabajo **Gitflow** para el registro de avances del diseño en un repositorio de GitHub.
- **IMPORTANTE:** Revise el detalle del caso Help Desk y la rúbrica en la guía técnica ubicada en el apartado de Recursos de la Unidad 1.



1. Introducción

Este trabajo tiene como finalidad diseñar la arquitectura lógica de un sistema Help Desk que apoye la gestión de incidentes técnicos dentro de un Data Center. La idea es que el sistema permita registrar, clasificar, asignar y dar seguimiento a problemas relacionados con hardware, red y software, facilitando el control de los fallos que puedan presentarse en la infraestructura de servidores.

Para el diseño se van a considerar los actores que participan en el proceso, los requerimientos del sistema, la creación de diagramas UML y la aplicación de algunos patrones de diseño de software. Además, se plantea utilizar GitHub con el modelo Gitflow para una fase posterior, con el fin de organizar mejor los avances del proyecto mediante ramas y commits.

Con esta propuesta se busca dejar una base clara para futuras etapas de desarrollo, aplicando principios de ingeniería de software que permitan que el sistema sea más fácil de mantener, escalar y controlar con el paso del tiempo.

2. Análisis del problema

En un Data Center pueden presentarse diferentes fallos técnicos relacionados con servidores, equipos de red, servicios internos o componentes de software. Si estos incidentes no se registran de forma ordenada, existe el riesgo de perder reportes, duplicarse solicitudes o retrasarse la atención por parte del personal técnico.

El sistema Help Desk propuesto busca solucionar este problema mediante la gestión de tickets. Cada ticket permitirá registrar el tipo de fallo, su descripción, prioridad, estado y técnico asignado. De esta manera, el personal de TI podrá dar seguimiento al incidente desde su creación hasta su resolución.

El sistema también permitirá clasificar los fallos en categorías como hardware, red y software, facilitando la asignación a técnicos especializados. Además, se conservará un historial de cambios para asegurar la trazabilidad del proceso y se podrán generar reportes que ayuden a evaluar la atención brindada dentro del Data Center.

3. Actores principales

Los actores principales del sistema son las personas que interactúan directamente con el Help Desk para reportar, atender y administrar incidentes técnicos dentro del Data Center.

Actor	Descripción	Funciones Principales
Usuario final	Persona que reporta fallos técnicos relacionados con los servicios o infraestructura del Data Center.	Crear tickets, categorizar fallos, adjuntar evidencia y consultar el estado del incidente.
Técnico de soporte	Personal especializado encargado de diagnosticar	Visualizar tickets asignados, diagnosticar fallos, actualizar estados,



	y resolver incidentes asignados.	registrar soluciones y escalar incidentes.
Administrador de sistemas	Responsable de gestionar usuarios, técnicos, categorías, asignaciones y reportes del sistema.	Gestionar usuarios, asignar técnicos, revisar historial, generar reportes y administrar categorías de fallos.

4. Requerimientos funcionales

Los requerimientos funcionales describen las acciones principales que el sistema Help Desk debe realizar para gestionar fallos técnicos dentro de un Data Center.

Código	Requerimiento funcional
RF01	El sistema permitirá iniciar sesión según el rol del usuario.
RF02	El usuario final podrá crear un ticket para reportar un fallo técnico.
RF03	El sistema permitirá categorizar el fallo como hardware, red o software.
RF04	El usuario final podrá agregar una descripción del incidente y adjuntar evidencias.
RF05	El sistema permitirá asignar una prioridad al ticket: baja, media, alta o crítica.
RF06	El administrador de sistemas podrá asignar un técnico de soporte especializado al ticket.
RF07	El técnico de soporte podrá visualizar los tickets que le han sido asignados.
RF08	El técnico podrá diagnosticar el incidente reportado.
RF09	El técnico podrá actualizar el estado del ticket: abierto, en proceso, escalado, resuelto o cerrado.
RF10	El técnico podrá registrar la solución aplicada al incidente.
RF11	El sistema enviará notificaciones cuando se cree o actualice un ticket.
RF12	El administrador de sistemas podrá consultar el historial de incidentes.
RF13	El administrador de sistemas podrá generar reportes de fallos atendidos, pendientes y resueltos.
RF14	El sistema permitirá escalar incidentes cuando el fallo requiera atención de mayor nivel.



Estos requerimientos ayudan a organizar el proceso de atención de incidentes desde el registro del fallo hasta su resolución, manteniendo control sobre técnicos, estados, prioridades e historial del sistema.

5. Requerimientos no funcionales

Los requerimientos no funcionales definen las características de calidad que el sistema Help Desk debe cumplir para funcionar correctamente en un entorno de Data Center.

Código	Requerimiento no funcional
RNF01	El sistema deberá proteger el acceso mediante autenticación de usuarios.
RNF02	El sistema deberá controlar permisos según el rol: usuario final, técnico de soporte y administrador de sistemas.
RNF03	El sistema deberá ser escalable para soportar más usuarios, técnicos e incidentes.
RNF04	El sistema deberá ser mantenible, permitiendo agregar nuevas categorías o funciones sin afectar todo el sistema.
RNF05	El sistema deberá mantener trazabilidad de los tickets mediante historial de cambios.
RNF06	El sistema deberá registrar evidencias y cambios importantes para auditoría.
RNF07	El sistema deberá responder de forma rápida al consultar o actualizar tickets.
RNF08	El sistema deberá mantener disponibilidad para reportar fallos cuando sea necesario.
RNF09	En una fase posterior, el repositorio del proyecto deberá reflejar trazabilidad de versiones mediante Gitflow.
RNF10	La información de incidentes deberá mantenerse íntegra y protegida contra modificaciones no autorizadas.

Estos requerimientos permiten que el sistema sea seguro, ordenado, escalable y fácil de mantener, aspectos importantes para una solución orientada a la gestión de fallos en un Data Center.

6. Diagrama de casos de uso

El diagrama de casos de uso representa las principales interacciones entre los actores y el sistema Help Desk para Data Center. En este caso participan tres actores: Usuario Final, Técnico de Soporte y Administrador de Sistemas.

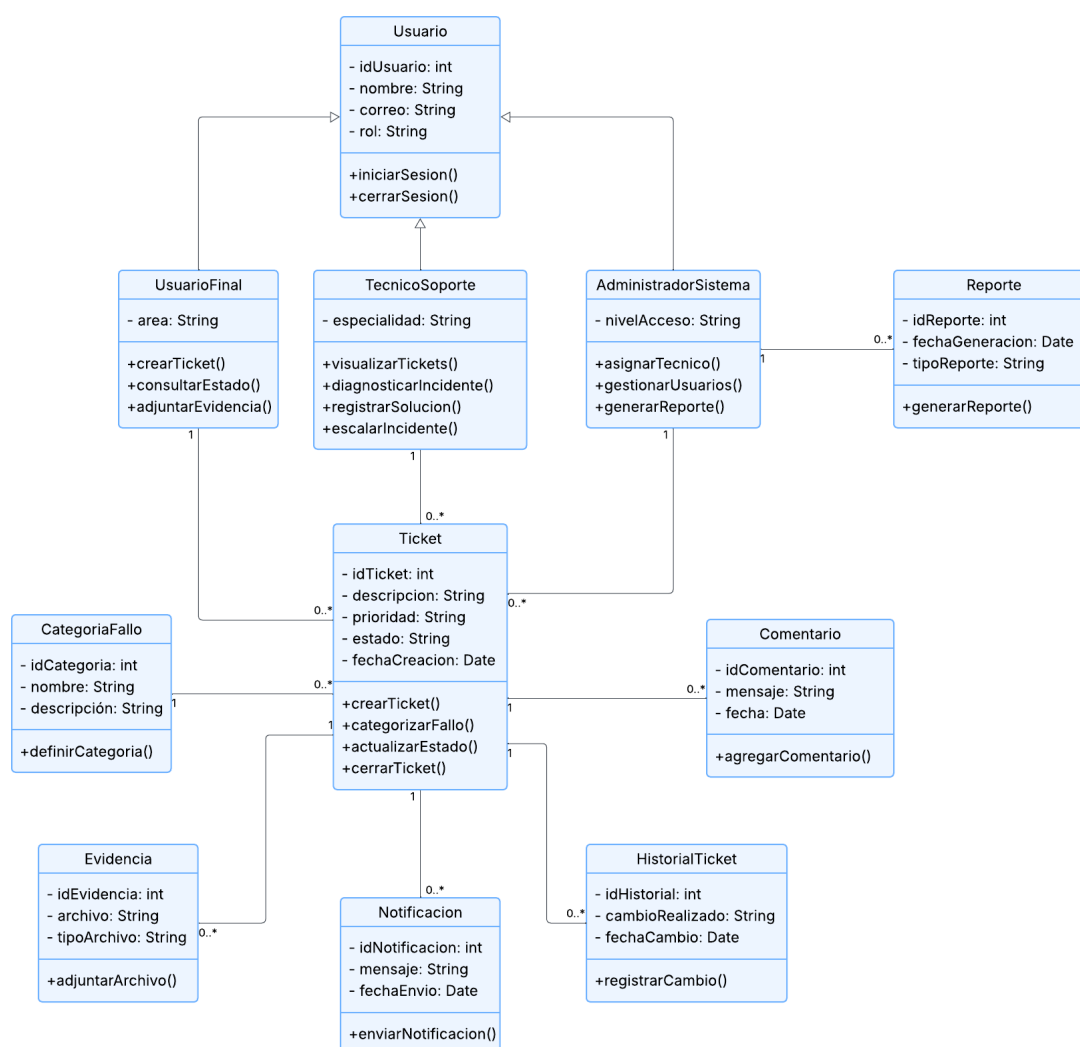


Actor	Casos de uso
Usuario Final	Iniciar sesión, crear ticket, categorizar fallo, adjuntar evidencia, consultar estado del ticket
Técnico de Soporte	Iniciar sesión, visualizar tickets asignados, diagnosticar incidente, actualizar estado del ticket, registrar solución, escalar incidente
Administrador de Sistemas	Iniciar sesión, gestionar usuarios, gestionar técnicos especializados, asignar técnico, gestionar categorías de fallo, generar reportes, revisar historial de incidentes

Este diagrama permite observar cómo el usuario final reporta fallos técnicos, el técnico de soporte atiende los incidentes asignados y el administrador de sistemas controla la gestión general del Help Desk.

7. Diagrama de clases UML

Figura 1. Diagrama de clases UML del Sistema Help Desk para Data Center.





El diagrama de clases representa la estructura lógica del sistema Help Desk para Data Center. La clase Usuario funciona como clase base para los roles UsuarioFinal, TecnicoSoporte y AdministradorSistema.

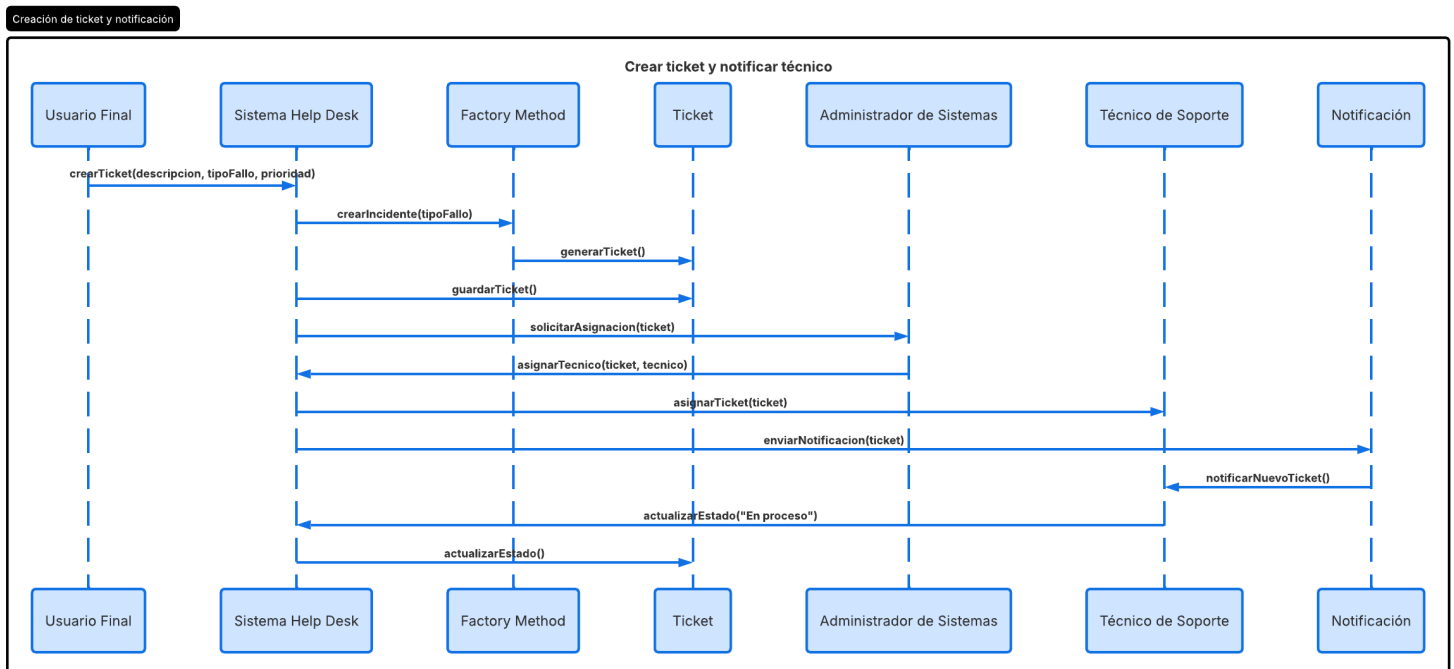
La clase Ticket es el elemento principal del sistema, ya que permite registrar, categorizar, actualizar y cerrar incidentes técnicos. Además, se relaciona con categorías de fallo, comentarios, evidencias, notificaciones e historial, permitiendo mantener la trazabilidad del proceso de atención.

8. Diagrama de secuencia

El diagrama de secuencia muestra el flujo de interacción entre los actores y los componentes principales del sistema durante la creación y atención inicial de un ticket.

En este caso se representa el proceso donde el usuario final reporta un fallo técnico, el sistema registra el ticket, lo clasifica, permite la asignación de un técnico y genera una notificación.

Figura 2. Diagrama de secuencia: Crear ticket y notificar técnico.



El diagrama de secuencia representa el proceso de creación y asignación de un ticket dentro del sistema Help Desk para Data Center. El usuario final registra un incidente, el sistema utiliza Factory Method para crear el tipo de ticket correspondiente según el fallo reportado, luego el administrador de sistemas asigna un técnico de soporte y el sistema genera una notificación. Finalmente, el técnico actualiza el estado del ticket a "En proceso", reflejando la atención inicial del incidente.

9. Arquitectura general del sistema

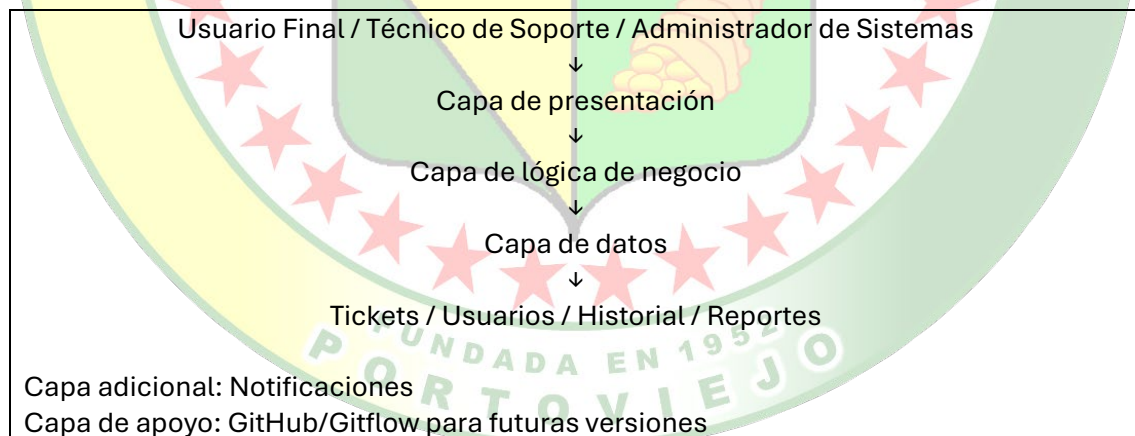
La arquitectura general del sistema Help Desk para Data Center se organiza por capas, con el objetivo de separar responsabilidades y facilitar el mantenimiento del sistema. Esta estructura permite que cada parte del sistema tenga una función específica dentro del proceso de gestión de incidentes.



La arquitectura propuesta se divide en las siguientes capas:

Capa	Descripción
Capa de presentación	Representa la parte con la que interactúan los usuarios del sistema. En una fase futura incluiría formularios para crear tickets, consultar estados y generar reportes.
Capa de lógica de negocio	Contiene las reglas principales del sistema, como crear tickets, categorizar fallos, asignar técnicos, actualizar estados y registrar soluciones.
Capa de datos	Se encarga de almacenar la información del sistema, como usuarios, tickets, categorías, evidencias, historial y reportes.
Capa de notificaciones	Gestiona los avisos enviados a los técnicos cuando se crea, asigna o actualiza un ticket.
Capa de control de versiones	Se propone el uso de GitHub y Gitflow para organizar los avances del proyecto en futuras fases.

Esta arquitectura permite que el sistema sea más ordenado, escalable y mantenible. Además, facilita la aplicación de patrones de diseño como Factory Method para la creación de tickets, Observer para las notificaciones y Singleton para la gestión centralizada del sistema.



La arquitectura propuesta separa las responsabilidades del sistema en capas, permitiendo organizar la interacción de los usuarios, la lógica de negocio, el almacenamiento de datos y el envío de notificaciones. Además, se considera GitHub/Gitflow como apoyo para el control de versiones en futuras fases del proyecto.



10. Patrones de diseño aplicados

Para mejorar la organización y mantenibilidad del sistema Help Desk para Data Center, se proponen patrones de diseño que permiten resolver problemas específicos dentro de la arquitectura del sistema.

Patrón	Tipo	Aplicación en el sistema
Singleton	Creacional	Se utiliza para mantener una única instancia del motor principal de gestión de tickets o de configuración del sistema.
Factory Method	Creacional	Se aplica para crear diferentes tipos de incidentes según la categoría del fallo: hardware, red o software.
Observer	Comportamiento	Se utiliza para notificar automáticamente al técnico de soporte cuando se crea, asigna o actualiza un ticket.
MVC	Arquitectural	Se propone para separar la interfaz, la lógica de negocio y los datos del sistema en futuras fases de implementación.

El patrón Singleton permite centralizar el acceso al motor principal del sistema, evitando múltiples instancias innecesarias. Factory Method facilita la creación de tickets según el tipo de fallo reportado, lo que permite extender el sistema si se agregan nuevas categorías. Observer ayuda en el módulo de notificaciones, ya que permite avisar automáticamente a los técnicos cuando ocurre un cambio importante en un ticket. Finalmente, MVC permite organizar el sistema en capas, separando la presentación, la lógica y los datos.

Estos patrones ayudan a que la arquitectura sea más ordenada, reutilizable y fácil de mantener, especialmente si el sistema se amplía en futuras etapas de desarrollo.

11. Uso propuesto de GitHub y Gitflow

En esta primera etapa, el trabajo se centra en el análisis y diseño lógico del sistema Help Desk para Data Center. Por ello, todavía no se desarrolla una implementación funcional completa ni una interfaz gráfica. Sin embargo, para una fase posterior se propone utilizar GitHub como herramienta de control de versiones, aplicando el modelo Gitflow para organizar los avances del proyecto.



La estructura de ramas propuesta sería:

- master: rama principal del proyecto.
- develop: rama de desarrollo donde se integran los avances.
- feature/analisis-problema: rama para el análisis del problema y requerimientos.
- feature/diagramas-uml: rama para los diagramas UML.
- feature/patrones-diseno: rama para documentar los patrones de diseño.
- feature/documentacion: rama para la elaboración del informe final.

El uso de Gitflow permitiría mantener un historial ordenado de cambios, separar los avances por secciones y facilitar la trazabilidad del proyecto durante futuras etapas de implementación.

Nota: El repositorio de GitHub será implementado en una fase posterior, según las indicaciones del docente.

12. Conclusiones

El diseño del sistema Help Desk para Data Center permitió identificar los elementos principales necesarios para gestionar incidentes técnicos relacionados con hardware, red y software. A partir del análisis del problema, los actores y los requerimientos, se estableció una base clara para organizar el funcionamiento del sistema.

Los diagramas UML elaborados ayudaron a representar tanto las interacciones de los actores como la estructura interna del sistema. El diagrama de clases muestra la relación entre usuarios, tickets, categorías, evidencias, notificaciones e historial, mientras que el diagrama de secuencia representa el proceso de creación y asignación de un ticket.

La aplicación de patrones de diseño como Singleton, Factory Method, Observer y MVC permite plantear una arquitectura más ordenada, reutilizable y fácil de mantener. Además, el uso propuesto de GitHub y Gitflow se plantea como apoyo para controlar versiones y organizar futuras etapas de implementación del proyecto.

13. Bibliografía

Atlassian. (s. f.). Gitflow Workflow. Atlassian Git Tutorial.

<https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>

Microsoft. (s. f.). Overview of ASP.NET Core MVC. Microsoft Learn.

<https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>

Object Management Group. (2017). Unified Modeling Language, Version 2.5.1. OMG.

<https://www.omg.org/spec/UML/2.5.1/About-UML>

Refactoring.Guru. (s. f.). Factory Method. <https://refactoring.guru/design-patterns/factory-method>

Refactoring.Guru. (s. f.). Observer. <https://refactoring.guru/design-patterns/observer>

Refactoring.Guru. (s. f.). Singleton. <https://refactoring.guru/design-patterns/singleton>